Report - BE-SHS

Project BE — Phase 1

Maximilien Larroche 22301310

---

1. Introduction

My project is to compare several data's structure of int in rust with the same trait. The first part is alone. I have to use the assemblist function : union, intersection, difference, symetric difference, also serialisation and deseralisation and fragmentation. The second half we have to compare the trait and result by group and decide one trait for everyone to continue.

2. My research

2.1. First Implementation

First of all, I don't know to code in rust, it was the most of my time. To learn I choose to watch video on Youtube and do the rust book. After that I start to build a trait with the basic function like new, add, remove, delete, there_is and also the protype of fragmenter. I start to implement the binarytree structure because it was the structure I know the most, it was difficult cause to the syntax of rust with the ownership system, mutable type and the gestion of references. In the process I discover that rust have a function drop that free the memory so the function delete wasn't necessary. I also add the prototype of map and an iterator.

After that I decide to implement the ensemblist function in the trait that I don't need to do it in every structure. I was able to do it with add, remove, there_is.

2.2. Test

I need to verify that my code is correct, to do it I just use a print command and see if the result was good. But I need to do it for every implementation and it's not a good way to do it. So I need to create a real test with comparason that can take any structure that implement the trait to verify the proper functioning.

2.3. Result

I have to test my code for the performance, in Rust there is a librairy call criterion that measure the time of a function. I decide to implement that in a generic way to not duplicate code for every structure that I test. The test is about 10 000 element and an sample of 50 test for the average. I implement HashSet and TreeSet who is already in the standar librairy, so it's really quick, it's for compare with mine structure.

With the test we see that mine structure is the worst because it's not balance, otherwise the HashSet et TreeSet are balance so the performance are better. The HashSet and TreeSet have very similaire performance.

3. Conclusion

To summerize the difficult part was to learn Rust and also understand the subject because my tutor don't respond to our mail, we don't have meeting, he had to give us a github link to start the project but we never saw it. To completely finish the first phase I have to complete the test and implement other structure. For the second phase, we need to create group and find a commun trait for everyone but for that we need to see our tutor to know how we do it.

```rust
pub trait StructureDonnee {
    fn new() -> Self;
    fn add(&mut self, value: i32);
    fn remove(&mut self, value: i32);
```

```
    fn there_is(&self, value: i32) -> bool;
    fn map(&self, f: impl Fn(i32) -> i32 + Copy) -> Self;
    fn iter(&self) -> Box<dyn Iterator<Item = &i32> + '_>;
    fn fragmenter(self, taille_max: usize) -> Vec<Self>
    where
        Self: Sized + Clone;
}
```

Opération	BST	HashSet	BTreeSet
add 10 000	1.05 ms	287 µs	284 µs
remove 10 000	1.35 ms	413 µs	413 µs
there_is	9.0 ns	7.3 ns	7.3 ns
Union	1.14 ms	145 µs	143 µs
Difference	1.35 ms	269 µs	270 µs
Diff. symetrique	33.4 ns	39.0 ns	39.6 ns
Intersection	1.14 ms	145 µs	147 µs
Fragmenter	2.30 ms	109 µs	109 µs

Table 1: Result of benchmarks — 10 000 elements